Analysis of Stochastic Compliance Failures in Governance Scaffolds and Deterministic Remediation Strategies for TASK.md Section 4

1. Introduction: The Constitutional Friction of AI Governance

The integration of Large Language Models (LLMs) into rigorous software governance frameworks presents a fundamental paradox: the collision between the probabilistic nature of generative AI and the binary, deterministic requirements of compliance auditing. In the Stela operating system, this friction is most acute at the "Contract Surface"—specifically, the TASK.md dispatch packet. This document serves as the operational nexus between the human Operator (Integrator) and the AI Agent (Contractor), governed by a constitution known as the *Policy of Truth* (PoT).[1] The failure of prompt-based methods to enforce the presence and integrity of the "Section 4" scaffold within this surface represents a critical vulnerability in the system's "Chain of Custody," threatening the validity of the "Proof of Work" (PoW) ledger.

The *Policy of Truth* establishes a severe operational doctrine: "The system prevents action that violates policy. It does not request compliance".[1] This axiom draws a sharp distinction between "Request" (a semantic interaction with an agent) and "Prevention" (a syntactic constraint by the environment). Current failures in maintaining the Section 4 scaffold—specifically the "Mandatory Closing Block" required for task closeout—stem from a reliance on the former. By delegating the structural maintenance of the dispatch packet to the stochastic agent via instructions in prompts.md, the system violates its own constitution, introducing "Ephemerality Risk" and "Drift" into the permanent record.[1]

This report provides an exhaustive analysis of this failure mode. It dissects the cognitive and architectural limitations of LLMs that lead to the truncation of trailing scaffolds, examines the specific "stances" defined in Stela's prompt engineering that exacerbate this issue, and formulates a rigorous, code-based solution. This solution, defined as the **Immutable Shell Protocol**, moves the responsibility for structural integrity from the probabilistic agent to the deterministic operating system, ensuring that the "Explainability Mandate" and "Linguistic Precision" required by the PoT are mechanically guaranteed rather than conversationally negotiated.

1.1 The Stela Filing Doctrine and the "Canon"

To understand the severity of the Section 4 failure, one must first operationalize the "Physical Laws" of the Stela repository. The PoT defines a strict **Filing Doctrine** that segregates the repository into functional domains, each with a specific governance role:

- **ops/ (Run):** The executable kernel. This layer contains binaries, manifests, and automation logic. It represents the "Physics" of the system.
- **docs/ (Explain):** The rationale layer. This contains manuals, specifications, and the constitution itself. It represents the "Law."
- **projects/ (Work):** The payload layer. This is where value is generated, but it is subordinate to the governance of ops/ and docs/.
- **storage/ (Trash):** A designated zone for ephemeral state, local artifacts, and trash.[1]

Within this topology, TASK.md is classified as a **Canon Surface**. It is not merely a text file; it is the **Active Contract**. It serves as the "Routing Surface" that directs the "Amnesiac Worker"—an AI agent that begins every session with zero context beyond what is loaded—through the execution cycle.[1] The integrity of TASK.md is paramount because it is the only bridge between the "Law" (PoT) and the "Work" (Projects).

1.2 The Evolution of Section 4: From Narrative to Metadata

The significance of Section 4 within TASK.md has evolved to address the specific weaknesses of LLM-driven development. Initially, task tracking utilized "Work Logs" or "Thread Transitions"—narrative sections intended to capture the history of a session. However, deployments such as **DP-OPS-0061** deprecated these sections due to "Ephemerality Risk".[1] Narrative logs were prone to hallucination, rewriting, or deletion during task resets, leading to "Invisible Work"—changes in the codebase that lacked a traceable audit trail.

The replacement, **Section 4.1: Mandatory Closing Block**, transformed the closeout process from a qualitative narrative into a quantitative metadata schema. It requires specific, machine-parseable fields:

- **Primary Commit Header:** A plaintext subject line for the Git history.
- **Pull Request Title:** A distinct plaintext title for the SCM interface.
- **Final Squash Stub:** A specific commit message for the merge event.
- **Extended Technical Manifest:** A precise list of impacted files.
- **Review Conversation Starter:** A markdown prompt for human-in-the-loop verification.[1]

This block is the "Cryptographic Receipt" of the session. Its presence allows the system to validate that the "Intent" (defined in Section 3) matches the "Outcome" (recorded in the Git log). When an LLM fails to generate or maintain this block, the "Loop of Truth" is broken. The work exists, but the proof does not.

2. The Pathology of Prompt-Based Enforcement

Failure

The reliance on "Prompt Engineering" to enforce the presence of Section 4 has proven fragile. Despite explicit instructions in the prompts.md file, the scaffold is frequently truncated, omitted, or malformed. This section analyzes the failure mechanisms through the lens of LLM architecture, token economics, and the specific dynamics of the Stela environment.

2.1 The Stochastic Decay of Instruction Adherence

Large Language Models operate as probabilistic token predictors. While they exhibit high fidelity to instructions at the beginning of a context window ("System Prompt Adherence"), this fidelity degrades as the generation length increases and the semantic distance from the original instruction grows.

2.1.1 Position Bias and the "Lost Tail"

Research into LLM behavior indicates a significant "Position Bias," often summarized as "Lost in the Middle," but equally critical is the "End-of-Sequence" bias.[2] When an agent is tasked with generating a comprehensive Dispatch Packet (DP), it focuses its "attention heads" on the primary logical task—the execution plan, the code patches, and the verification steps (Section 3).

In the Stela workflow:

1. The agent receives a "Stance" instruction (e.g., Architect) to "include the empty Section 4 scaffold".[1]
2. The agent generates Section 3.1 (Freshness), 3.2 (Context), 3.3 (Scope), and 3.4 (Execution). This is a high-entropy, high-complexity generation task.
3. By the time the agent completes Section 3.4.5 (Receipts), it has satisfied the *semantic* requirements of the prompt ("Draft a plan").
4. The probability of generating a "stop token" (indicating completion) spikes. The instruction to append a static, empty footer (Section 4) competes with the model's inherent bias to conclude the generation once the "intellectual work" is done. The footer is statistically pruned because it carries low semantic weight compared to the execution plan.

2.1.2 The "Partial Completion" Phenomenon

LLM agents struggle with multi-step workflows that mix creative generation with rote formatting. This "Partial Completion" problem [4] is exacerbated when the rote task is placed at the very end of a long chain of reasoning. The model prioritizes the "reasoning" steps (generating the plan) and often drops the final "formatting" step (appending the scaffold), effectively hallucinating that the user implies the standard footer or simply forgetting it due to context saturation.

2.2 The Paradox of the "Explainability Mandate"

The Stela system currently operates under an **Explainability Mandate**, also known as the **Doctrine of Verbosity**. This mandate requires a 25% increase in documentation density to achieve "Resolution".[1] While strategically sound for governance, this mandate mechanically antagonizes the LLM's ability to maintain Section 4.

2.2.1 Verbosity as Context Saturation

The "Deep Spec" protocol requires every tool in docs/ops/specs/ to be documented with "Theological Anchors," "Mechanical Exegesis," and "Forensic Value".[1] When a session starts, the OPEN prompt loads these verbose contexts.

- **Result:** The context window is filled with high-density, "high-resolution" text regarding the *why* and *how* of the system.
- **Displacement:** The specific instruction to "maintain Section 4" becomes a tiny fraction of the total token count. As the ratio of "Context" (rationale) to "Instruction" (formatting) skews heavily toward context, the model's adherence to the formatting constraint weakens. The "Law" (PoT) obscures the "Form" (TASK.md).

2.2.2 The Cost of "Resolution"

The mandate defines "Verbosity as Resolution".[1] High-resolution instructions (e.g., "Execute tools/test/agent.sh...") consume more tokens than low-resolution ones ("Run tests"). This accumulation of tokens pushes the generation closer to the model's output limits (max tokens). If a generation hits the token limit, the content is truncated. Since Section 4 is the final block, it is the first to be sacrificed to the token limit, leaving the file in a "Drifted" state.

2.3 Analysis of "Stance" Instructions in prompts.md

A forensic review of the prompts.md file reveals that the instructions for Section 4 are structurally weak because they rely on the "cooperation" of the model rather than a physical constraint.

2.3.1 The "Hygiene" Stance

The **Hygiene** stance is designed to conform DPs to the standard.[1]

- **Instruction:** "Output the full Dispatch Packet (including the empty Section 4 scaffold from TASK.md at the bottom)."
- **Failure Mode:** This instruction relies on the model *retrieving* the scaffold from TASK.md and *reproducing* it perfectly. If the model hallucinates a variation (e.g., changing "Primary Commit Header" to "Commit Message"), the strict regex validators in the downstream CI pipeline will fail. The prompt relies on "Recall," which is stochastic, rather than "Injection," which is deterministic.

2.3.2 The "Architect" Stance

The **Architect** stance drafts new DPs.[1]

- **Instruction:** "Strictly follow TASK.md template."
- **Failure Mode:** "Strictly" is a semantic intensifier, not a functional constraint. The model may interpret "follow template" as "follow the general structure," leading to subtle deviations in the Section 4 labels (e.g., using Markdown headers ### instead of plaintext labels, or omitting the (plaintext) type hints). These deviations render the block unparseable by tools/lint/dp.sh.

2.4 Inadequacy of Passive Linting

The existing linting infrastructure detects the absence of Section 4 but fails to correct it, leading to a "Stop Work" condition that requires human intervention—a violation of the "Agentic" ideal.

2.4.1 tools/lint/task.sh

This script acts as the "Sole TASK surface/container schema enforcer".[1]

- **Mechanism:** It parses TASK.md looking for # 4. Closeout.
- **Limitation:** It is a **passive gate**. It returns Exit 1 if the section is missing.[1] It does not repair the file. In an agentic loop, an Exit 1 stops the pipeline. The agent receives an error message ("Section 4 missing") and attempts to regenerate it. However, because of the "Position Bias" and "Context Saturation" described above, the agent often fails repeatedly or hallucinates a malformed fix, entering a "Doom Loop" of validation failures.[4]

2.4.2 tools/lint/dp.sh

This script validates the "Payload" (Section 3).[1]

- **Mechanism:** It extracts content between headers.
- **Limitation:** It assumes the container is intact. If Section 4 is missing, the extraction logic (often based on awk ranges) may fail catastrophically or bleed into the end of the file, corrupting the validation of Section 3.

3. The Deterministic Solution: The Immutable Shell Protocol

To resolve these failures, Stela must transition from **Prompt Engineering** (asking the model to comply) to **Systems Engineering** (forcing the model to comply). We propose the **Immutable Shell Protocol**.

This protocol treats TASK.md not as a single mutable document, but as a composite object

consisting of:

1. **The Shell (Immutable):** Sections 1, 2, and 4. These are system-managed constraints.
2. **The Payload (Mutable):** Section 3. This is the only area the agent is permitted to touch.

3.1 Mechanism 1: Canonical Template Injection

We must establish a "Golden Master" for Section 4 that is machine-readable and injection-ready. Reliance on the agent to "read TASK.md and copy Section 4" is circular and error-prone.

Artifact: docs/ops/specs/surfaces/task.md (Already identified as "Local task template" in .gitignore [1]).

Action: We formalize the content of this file to include explicit delimiters that allow sed/awk to surgically extract the scaffold.

4. Closeout

- Execute docs/MANUAL.md Closeout Cycle in order (Verify, Harvest, Refresh, Log, Prune).
- Update SoP.md and PoW.md with DP entries, including objective summary and verification commands run.
- Protocol order for closeout: Verify -> Generate Results -> COMMIT (Operator Only) -> Prune.
- Run prune hygiene at the end of the closeout sequence:./ops/bin/prune --dp=DP-OPS-XXXX --scrub.
- Use./ops/bin/prune --reset-task only for explicit TASK baseline reset after PoW entry exists for the active DP id.
- Ensure the next session begins with refreshed session artifacts and matching receipts.

4.1 Mandatory Closing Block

Primary Commit Header (plaintext)

Pull Request Title (plaintext)

Pull Request Description (markdown)

Final Squash Stub (plaintext) (Must differ from #1)

Extended Technical Manifest (plaintext)

Review Conversation Starter (markdown)

3.2 Mechanism 2: The ops/bin/open Injection Hook

The ops/bin/open binary controls the "Freshness Gate".[1] It is the start of every session. Currently, it generates a prompt. It must be refactored to perform **Context Engineering**—physically manipulating the workspace before the agent enters.

Architectural Logic:

1. **Pre-Flight:** Before generating the OPEN artifact, ops/bin/open calls a new helper function: ensure_scaffold_integrity.
2. **Extraction:** This function reads docs/ops/specs/surfaces/task.md and extracts the block between TASK-TEMPLATE-SECTION-4-BEGIN and END.
3. **Sanitization:** It reads the live TASK.md and strips any content appearing after the end of Section 3 (effectively deleting any malformed or missing Section 4).
4. **Injection:** It forcibly appends the canonical Section 4 block to TASK.md.
5. **Verification:** It calculates a checksum of the appended block to guarantee integrity.

Result: The agent *never* sees a TASK.md without a perfect Section 4. The scaffold is present before the agent attempts to read or write.

3.3 Mechanism 3: The Self-Healing Linter (tools/lint/task.sh)

We transform tools/lint/task.sh from a passive validator into an active remediation tool. This breaks the "Doom Loop" of agent retry failures.

Current Logic:

Bash

```
if! grep -q "^## 4. Closeout" "$FILE"; then
  echo "FAIL: Missing Section 4"
  exit 1
fi
```

Proposed Logic (Self-Healing):

Bash

```
if! grep -q "^## 4. Closeout" "$FILE"; then
  echo "WARN: Section 4 missing or corrupt. Initiating auto-repair..."
```

```
  # Call the scaffold injector
  source ops/lib/scripts/enforce_scaffold.sh
  inject_section_4 "$FILE"

  # Re-verify
  if grep -q "^## 4. Closeout" "$FILE"; then
    echo "FIX: Section 4 restored from canon."
    exit 0 # Pass the lint, as the file is now correct
  else
    echo "CRITICAL: Auto-repair failed."
    exit 1
  fi
fi
```

This ensures that "Drift" is instantly corrected by the system, satisfying the PoT axiom: "Action: deception or incoherence are inherently destabilizing and must be resolved".[1] The resolution becomes automatic.

3.4 Mechanism 4: Prompt Instruction Refactoring

With the deterministic shell in place, the instructions in prompts.md must be updated to reflect the new division of labor. We remove the cognitive load of "creating" the scaffold.

Revised Prompt Instruction:

> "Focus exclusively on Section 3 (The Payload). Do not generate Section 4. The system automatically enforces the Section 4 scaffold. Any attempt to modify Section 4 will be overwritten by the kernel."

This eliminates the "Position Bias" risk because the model is no longer tasked with generating the tail of the document.

4. Implementation Specification

This section provides the "Mechanical Exegesis" required by the 25% verbosity mandate, detailing the specific code required to implement the solution.

4.1 New Artifact: ops/lib/scripts/enforce_scaffold.sh

This script centralizes the logic for extracting and injecting the scaffold. It is the "Theological Anchor" for the "Prevention" axiom.

Component	Specification

Path	ops/lib/scripts/enforce_scaffold.sh
Purpose	Deterministic extraction and injection of the Section 4 scaffold.
Inputs	TASK_FILE (Target), TEMPLATE_FILE (Source)
Logic	1. sed to extract lines between template delimiters. 2. sed to delete lines from ## 4. Closeout to EOF in target. 3. cat to append extracted template to target.
Safety	Must verify that TEMPLATE_FILE exists and contains valid delimiters before touching TASK_FILE.

Code Implementation:

Bash

```bash
#!/usr/bin/env bash
# ops/lib/scripts/enforce_scaffold.sh
set -euo pipefail

inject_scaffold() {
  local target="$1"
  local template="${REPO_ROOT}/docs/ops/specs/surfaces/task.md"

  # 1. Extract Canonical Scaffold
  local scaffold
  scaffold=$(sed -n '//,//p' "$template" | sed '1d;$d')

  if [[ -z "$scaffold" ]]; then
```

```
        echo "ERROR: Could not extract Section 4 scaffold from ${template}" >&2
        return 1
    fi

    # 2. Truncate Target
    # Remove everything from ## 4. Closeout onwards
    if grep -q "^## 4. Closeout" "$target"; then
        sed -i '/^## 4. Closeout/,$d' "$target"
    fi

    # 3. Inject Scaffold
    # Ensure newline separation
    echo "" >> "$target"
    echo "$scaffold" >> "$target"

    return 0
}
```

4.2 Patching tools/lint/dp.sh

The current extract_dp_payload function in dp.sh likely grabs the entire file or uses simple whitespace trimming.[1] It must be updated to strictly isolate Section 3, preventing it from accidentally validating (or failing on) the auto-injected Section 4.

Revised Logic:

Bash

```
extract_dp_payload() {
  local source_path="$1"
  # Extract only lines starting from "## 3." until "## 4."
  # This isolates the AI's work domain from the System's domain.
  awk '
    /^## 3\./ { in_payload=1; print; next }
    /^## 4\./ { in_payload=0; exit }
    in_payload { print }
  ' "$source_path"
}
```

4.3 Updating the "Mandatory Closing Block" Schema

Table 1 defines the strict schema that the system will now deterministically enforce.

Table 1: Mandatory Closing Block Schema (Section 4.1)

Field	Type	Validation Rule	Purpose
Primary Commit Header	plaintext	Non-empty, no markdown formatting	Maps to git commit -m subject line.
Pull Request Title	plaintext	Non-empty, no markdown formatting	Maps to GitHub PR title.
Pull Request Description	markdown	Non-empty, allows markdown	Maps to GitHub PR body.
Final Squash Stub	plaintext	Must differ from Primary Commit Header	Ensures distinct squash message history.
Extended Technical Manifest	plaintext	List of file paths	Must match git diff --name-only.
Review Conversation Starter	markdown	Non-empty question/prompt	Facilitates human-in-the-loop audit.

5. Strategic Implications and "Context Engineering"

The shift from prompt-based to code-based enforcement represents a strategic evolution in the Stela governance model: the move from **Prompt Engineering** to **Context Engineering**.

5.1 Redefining "Relatability" through Determinism

While "Relatability" in UI typically refers to human-centric design, in the context of the Stela "Amnesiac Worker," relatability refers to the **predictability** of the environment.[1] An environment that behaves deterministically (e.g., "The door always closes itself") is more relatable to an agent than one that requires constant negotiation ("Please close the door").

- **Anecdotal Grounding:** The "story" of the repository changes. Instead of "The agent forgot the footer," the story becomes "The system secured the ledger." This aligns with the "Deep Spec" requirement for **Forensic Value**.[1] The presence of Section 4 becomes an axiom, not a variable.

5.2 Token Economics and the Verbosity Budget

By offloading the structural overhead of Section 4 to the deterministic layer, we reclaim context window capacity.

- **Savings:** The tokens previously reserved for generating the boilerplate footer can now be allocated to the **Mechanical Exegesis** of the task itself.
- **Alignment:** This supports the **Explainability Mandate**. We are not reducing the total information; we are simply moving the *source* of that information from the expensive/fragile "Neural" layer to the cheap/robust "Symbolic" layer (code).

5.3 Addressing "Bias against Extralegal Sections"

External research [3] suggests that LLMs exhibit bias against "extralegal" or trailing sections of text, viewing them as less relevant than the core "legal" text (the code). By making Section 4 a "System Object" rather than a "Generated Object," we bypass this bias entirely. The LLM does not need to value Section 4; the OS values it for it.

6. Conclusion: Restoring the Verifiable Equilibrium

The failure of the TASK.md Section 4 scaffold was not a failure of prompting, but a failure of architecture. Attempting to enforce strict compliance through probabilistic requests violates the core axiom of the *Policy of Truth*: "The system prevents action that violates policy."

By implementing the **Immutable Shell Protocol**, we effectively "bracket" the AI's probabilistic creativity within deterministic guardrails. The AI is free to innovate within Section 3, but the system rigorously enforces the boundaries of Sections 1, 2, and 4. This restores the **Verifiable Equilibrium** between the "Run" layer and the "Explain" layer, ensuring that every unit of work produces a cryptographic receipt that is immune to stochastic decay, hallucination, or laziness. The "Contract Surface" is no longer a request; it is a law.

Table 2: Comparison of Enforcement Models

Feature	Prompt-Based (Legacy)	Code-Based (Immutable Shell)
Enforcement Mechanism	prompts.md instructions ("Stance")	ops/lib/scripts/enforce_scaffold.sh

Reliability	Stochastic (Probabilistic)	Deterministic (Binary)
Failure Mode	Truncation, Hallucination, Omission	Permission Errors (Rare)
Drift Resolution	Manual Intervention (Human)	Auto-Correction (System)
Context Cost	High (Verbose instructions required)	Zero (Handled by OS)
PoT Compliance	Violates "Prevention" Axiom	Enforces "Prevention" Axiom

Report generated by the Principal Systems Architect for Stela Governance Operations.

Works cited

1. dump-full-main-6043fa23.txt
2. Unpacking the bias of large language models | MIT News, accessed February 14, 2026, https://news.mit.edu/2025/unpacking-large-language-model-bias-0617
3. Eliminating Position Bias of Language Models: A Mechanistic Approach - OpenReview, accessed February 14, 2026, https://openreview.net/forum?id=fvkElsJOsN
4. Tackling the “Partial Completion” Problem in LLM AI Agents | by George Karapetyan | Medium, accessed February 14, 2026, https://medium.com/@georgekar91/tackling-the-partial-completion-problem-in-llm-agents-9a7ec8949c84
5. How do you stop LLM from looping when it can't solve the issue? - Reddit, accessed February 14, 2026, https://www.reddit.com/r/ChatGPTCoding/comments/1go81so/how_do_you_stop_llm_from_looping_when_it_cant/
6. Not ready for the bench: LLM legal interpretation is unstable and out of step with human judgments - arXiv, accessed February 14, 2026, https://arxiv.org/html/2510.25356v1